Library Management System: Project Brief for Google AI Studio

College project · Version 1.0 · October 22, 2026

How to use this brief: Paste everything from "Part 1" onward at the start of your AI Studio session, and re-paste it whenever you start a fresh chat. Then build one feature at a time using the build prompts. If a decision changes, update this document first, then tell the AI.

Part 1: Project overview

Build a web-based **Library Management System** for a small college library. It maintains records of books, authors, categories, student members, book issue and return transactions, and fines for late returns. It is a college project, so the priority is a **reliable, demo-ready app** with clean, readable code that can be explained and tested.

There are two kinds of users: the **Librarian** and **Students**. The system is a responsive web app (desktop, tablet, mobile) that works in modern browsers (Chrome, Firefox, Safari, Edge) without plugins.

Part 2: Users and roles

Librarian (full access)

- Manages books, authors and categories
- Registers and updates student members
- Issues books and records returns
- Views and collects fines
- Searches the catalogue

Student (limited, read-only access to own data)

- Searches the catalogue and sees availability
- Views their own currently issued books, due dates and history
- Views their own fines and payment status
- Cannot see other students' data and cannot perform any librarian action

Account setup

- Librarian accounts are created manually by the project owner in the Supabase dashboard (one seeded librarian is enough).
- The librarian registers a student with their email. The student then signs up with that same email, and the system automatically links the login to the member record. If the email is not registered by the librarian, sign-up must not grant access to anything.

Part 3: Tech stack and architecture

- **Frontend:** Responsive web app (React with Tailwind CSS is fine), built in Google AI Studio.
- **Backend and database:** Supabase (Postgres) with Supabase Auth. One database, multiple tables.
- **Security:** Row Level Security (RLS) must be enabled on every table. Roles are enforced at the database level, not just by hiding buttons.
- **Business logic:** Issue, return and fine payment must be implemented as **Postgres functions called via RPC**, so each runs as one atomic transaction and can be tested directly.
- **Keys:** The frontend uses only the Supabase URL and the public anon key, stored in environment variables. **Never use or expose the service role key in frontend code.**
- **Offline:** The app is a PWA. The latest catalogue (books, authors, categories, availability) is cached on the device so that search works offline, with a visible "Offline: showing data last synced at [time]" banner. All write actions (issue, return, pay, add, edit, delete) are disabled offline with a clear message. No offline writes.

Part 4: Data model

Use these tables. Names can be adjusted for convention, but keep the relationships and constraints.

authors

- id (primary key), name

categories

- id (primary key), name (unique)

books

- id (primary key), isbn (unique), title, author_id (foreign key to authors), category_id (foreign key to categories), publisher
- total_copies (integer, at least 0), available_copies (integer, at least 0 and at most total_copies)

members

- id (primary key), member_code (unique, human-readable such as M001), full_name, email (unique), phone, membership_type, status (Active or Inactive)
- auth_user_id (nullable, set when the student signs up with the matching email)

profiles

- user_id (links to the auth user), role (librarian or student)

transactions

- id (primary key), book_id, member_id, issued_by (librarian user id)
- issue_date, due_date, return_date (null until returned), status (Issued or Returned)

fines

- id (primary key), transaction_id, member_id, days_late, amount
- status (Unpaid or Paid), generated_on, paid_on (null until paid), collected_by (librarian user id)

Part 5: Business rules (these are the source of truth)

- **BR-1 Loan period:** The due date is the issue date plus 14 days, set automatically.
- **BR-2 Fine calculation:** Fine = days late × ₹5, with no cap. Days late is the return date minus the due date, counted in whole days. A book returned on or before the due date is on time and has no fine. A fine record is created automatically at return only if the fine is above ₹0.
- **BR-3 Borrow limit:** A member can have at most 5 books issued (status Issued) at one time.
- **BR-4 Issue is blocked if any of these is true:** the book has 0 available copies, the member's status is not Active, the member has any Unpaid fine, or the member already has 5 issued books. The error message must say which reason applies.
- **BR-5 Stock:** Issuing decreases available_copies by 1, and returning increases it by 1, in the same atomic step as the transaction update. Available copies must never go below 0 or above total_copies, even if two librarians act at the same time.

- **BR-6 Returns:** Only the librarian records returns. Only a transaction with status Issued can be returned. Returning something already returned must be rejected with a clear message.
- **BR-7 Fine payment:** Full payment only. The librarian marks an Unpaid fine as Paid, which sets paid_on and collected_by. Partial payments do not exist.
- **BR-8 Deletion protection:** A book or member that has any active (unreturned) transaction cannot be deleted. total_copies cannot be set lower than the number of copies currently issued.
- **BR-9 Search:** Case-insensitive, partial-text match across title, author name, category name and ISBN. Results show availability.
- **BR-10 Unavailable books:** If a book has 0 available copies, show the earliest expected return date, which is the earliest due date among that book's active transactions.
- **BR-11 Dates and time:** All dates use India time (Asia/Kolkata). Compare dates as whole days, not timestamps, to avoid off-by-one errors.
- **BR-12 Slips:** After a successful issue, return, or fine payment, show an on-screen slip (member, book, dates, fine details where applicable, slip date) that can be printed or saved as PDF.

Part 6: Features mapped to requirements

- **FR-1** Librarian adds, updates and removes books, authors and categories (subject to BR-8).
- **FR-2** Librarian registers new student members and updates their contact and membership details.
- **FR-3** Librarian issues books to members. The transaction is linked to the Member ID and the due date is set automatically (BR-1).
- **FR-4** Librarian records returns. Return date and transaction status update automatically (BR-6).
- **FR-5** A fine is automatically calculated and recorded when a book is returned late (BR-2).
- **FR-6** Librarian accepts fine payments and marks them Paid (BR-7).
- **FR-7** Available copies update automatically on issue and return (BR-5).
- **FR-8** Issuing is blocked under the conditions in BR-4.
- **FR-9** Librarians and students search by title, author, category or ISBN (BR-9).

- **FR-10** Secure login with username (email) and password before accessing any dashboard. Passwords are hashed by Supabase Auth and never stored or shown as plain text.

Part 7: Screens

Follow a clean, simple style similar to the project wireframes (card-based forms, clear primary buttons).

- **Login:** Email, password, Log In button. A separate "Student first-time sign-up" option for students already registered by the librarian.
- **Librarian dashboard:** Quick stats (books, members, currently issued, overdue, unpaid fines) and navigation.
- **Catalogue search (everyone):** Search box and results table (title, author, category, availability). Unavailable books show the earliest expected return date.
- **Manage catalogue (librarian):** Books, authors and categories with add, edit and delete.
- **Members (librarian):** Registration form (full name, contact number and email, membership type) and a list with edit and status change.
- **Issue book (librarian):** Book ID or ISBN, Member ID, a Check Availability button showing status, and Confirm Issue. Show specific error messages for each blocked case.
- **Return book (librarian):** Find an active issue by member or book, confirm return, and show the fine if one applies.
- **Fines (librarian):** List of fines filtered by Unpaid and Paid, with a Mark as Paid action and slip.
- **My account (student):** Currently issued books with due dates (overdue highlighted), history, and fines with status.
- **Slips:** Printable issue, return and fine slips.
- **All forms:** Show validation messages for empty or invalid input, and error, loading and empty states.

Part 8: Non-functional requirements

- **Performance:** Searching the catalogue and issuing a book complete in under 2 seconds.

- **Security:** Hashed passwords, RLS on all tables, students can only read their own member, transaction and fine data.
- **Usability:** Intuitive enough for a new librarian to learn in under 30 minutes.
- **Compatibility:** Works in Chrome, Firefox, Safari and Edge without plugins.
- **Maintainability:** Clean structure, comments on non-obvious logic, and a SCHEMA.md file describing every table, column and function.
- **Reliability, availability and capacity** (daily backups, 99% uptime, 50,000 books and 10,000 members): documented as design targets and tied to the hosting plan. The schema should use indexes on search fields (title, isbn, member_code) to support them.

Part 9: Out of scope

Reservations, renewals, partial fine payments, offline writes, email or SMS notifications, and online payment gateways. Do not add these unless asked.

Part 10: Seed data for demos

Provide a SQL seed script using only **fictional** data:

- About 30 books across at least 5 categories and several authors, with varied copy counts, including one book with 0 available copies
- 10 members, including one Inactive member and one member with 5 books issued
- A few active transactions, including 2 or 3 overdue ones
- One Unpaid fine and one Paid fine
- 1 librarian account and 2 student accounts linked to members

Part 11: Working rules for the AI

- Build **only the feature requested** in each prompt. **Do not change, rename or remove any existing feature** unless explicitly told to.
- After each change, give a short summary: files changed, new tables or functions, and anything that needs manual setup.
- Put all business rules (BR-1 to BR-12) in database functions or constraints, not only in the frontend.
- Never place secrets or the service role key in code. Use environment variables.

- Keep code readable with simple names and short comments, because the owner must explain it in a viva.
- If a requirement is ambiguous or conflicts with this brief, **ask a question instead of guessing.**
- Keep SCHEMA.md up to date as the schema changes.

Part 12: Build order

1. Foundation: project structure, Supabase connection, database schema with RLS, login with the two roles
2. Book, author and category management
3. Member management and student account linking
4. Search and filters (with availability and earliest return date)
5. Issue book (all blocking rules, stock update, slip)
6. Return book (fine calculation, stock update, slip)
7. Fines list and payment (slip)
8. Student "My account" view
9. Dashboard and reports
10. PWA and offline catalogue search
11. Polish: validation, error states, responsive layout, seed data, SCHEMA.md